

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

Brad Scott

sctbra008@myuct.ac.za
University of Cape Town
Cape Town, South Africa

Abstract

Reinforcement learning has been increasingly applied to competitive environments in which an opponent’s behaviour is not fixed. When an opponent’s behaviour changes abruptly, the resulting non-stationarity can weaken a previously effective policy. Hierarchical architectures are broadly expected to re-adapt faster than flat architectures under these circumstances, yet this expectation has barely been tested specifically against a shifting opponent. This work investigates whether a hierarchical Feudal Network (FuN) recovers more quickly than a flat Proximal Policy Optimization (PPO) agent following abrupt opponent strategy shifts in the game of Go. Both agents are trained against fixed heuristic opponents in PettingZoo’s go_v5 on 9×9 and 13×13 boards, after which mid-training opponent shifts of varying magnitude and frequency are introduced. The effect of these shifts is measured by recovery time, along with complementary measures of depth and overall cost of the consequent performance drop. Across the conditions examined, the hierarchical agent does not recover faster than the flat agent. These findings suggest that the FuN hierarchy does not provide the expected advantage under opponent-induced non-stationarity, indicating that a flat policy is the more reliable choice when the opponent changes rather than the goal.

1 Introduction

Reinforcement learning (RL) is a framework in which an agent learns how to behave by interacting with an environment [19]. The agent observes different states, takes actions on those states and receives a reward. From this the agent gradually adjusts its behaviour to maximise these rewards it collects over time. Through repeated trial and error the agent learns a policy, a function that maps each state to an action [19]. A policy works well as long as the environment stays the same but many real settings are not static. In competitive games, an opponent does not play the same way forever and may change its strategy as the game unfolds. When the opponent changes, the conditions the agent originally learned against no longer hold. Behaviour that was once effective can suddenly perform poorly. The effect of a shifting environment caused by an opponent’s behaviour changing, is known as non-stationarity [13]. This is the central difficulty the study addresses. Abrupt opponent strategy shifts arise in any competitive game-playing setting and an agent that cannot adjust quickly suffers a continuing fall in performance that leaves it vulnerable until it recovers. This motivates investigating how the architecture of a learning agent affects how quickly it recovers once an opponent shifts strategy.

A learning agent is commonly implemented as a flat policy, which is a single network that maps observations directly to actions [2]. A widely used example of this design is Proximal Policy

Optimization (PPO). When the opponent shifts, a flat agent has to revise the single policy it learned as a whole. This is slow and risks overwriting behaviour that is still useful. An alternative design is a hierarchical agent, which splits decision-making across levels. The Feudal Network (FuN) [21] is one instance of this structure. The architecture has a high-level manager setting goals over a longer timescale while a low-level worker chooses the primitive actions that pursue them. The idea this research tests is that the hierarchy should recover faster after an opponent changes strategy. The manager can switch to a new high-level plan while the worker keeps the low-level skills it has already acquired. This minimizes relearning compared to a flat agent. However, this advantage has not been demonstrated with an opponent that changes strategy during game-play.

1.1 Research Question

This gap motivates the paper’s research question: *Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?* The question is studied in the board game Go, where a hierarchical Feudal Network and a flat PPO agent are each trained against fixed heuristic opponents and then undergo abrupt strategy shifts on two board sizes, 9×9 and 13×13 .

The study is built around three main elements. First, a controlled benchmark is constructed for measuring recovery from opponent strategy shifts in Go. The timing, magnitude and frequency of each shift are fixed in advance so that recovery can be measured cleanly. Second, a Feudal Network and a flat PPO agent are compared under identical conditions, matched across shift magnitude, frequency and board size. Any difference in recovery reflects the architecture rather than the experimental setup. Third, recovery is assessed with three supporting metrics, each capturing a different part of the process.

The focused elements the study is built around supplies a direct test of how quickly a learning agent recovers a policy once a shift in an opponent’s strategy has made it ineffective. It establishes whether the recovery advantage expected of the hierarchy holds when a disruption is introduced into the learning environment. The findings of the research showed that this advantage did not materialise, instead favouring the flat baseline in adaptation capability. The sections that follow develop this result and examine why it arises.

The remainder of the paper is organised as follows. Section 2 reviews related work on non-stationarity, flat and hierarchical reinforcement learning and on the measurement of adaptation and recovery. Section 3 sets out the design and implementation, which covers the environment and opponents, the two agents, the experimental design, the recovery metrics and tools used. Section 4

presents the results for both agents across all conditions. Section 5 discusses what the results reveal about the role of hierarchy in recovery. Section 6 concludes and outlines directions for future work.

2 Related Work

How far performance falls and how quickly it recovers depends on the nature of the opponent’s change and on how the agent itself is organised. This section reviews the research behind this. It moves from reinforcement learning in Go, to how a shifting opponent causes non-stationarity, to flat and hierarchical reinforcement learning as well as the PPO and Feudal Network agents this study compares. It finishes with how adaptation and recovery have been measured, outlining the metrics used and the gap this work addresses.

2.1 Reinforcement Learning in Go

Go is an ancient two-player board game that originated in China [16]. Players take turns placing black and white stones on the intersections of a square grid. Both players see the whole board at all times, so no information is hidden. A stone is captured once opposing stones occupy every point beside it. The winner is whoever controls the larger share of the board at the end. This is decided by area scoring, which counts both territory and stones in play [16]. Moving first is an advantage so to balance the game a fixed number of points called *komi* is added to the second player’s score. A game continues until both players pass in succession, typically after a few hundred moves [20].

Although its rules are simple, Go is extremely hard to play well and for many years this made it a landmark AI challenge. Each turn offers a large number of legal moves and a full game runs hundreds of turns, giving Go a far larger space of possible games than chess [16]. The search-based programs that had already mastered chess could not cope with this scale [4], so progress instead came from reinforcement learning.

The breakthrough came with AlphaGo, which combined neural networks with look-ahead search and millions of self-play games to beat a professional human player [16]. This drew broad attention to reinforcement learning. Systems like AlphaGo, however, trained under stable conditions improving against themselves or fixed opponents rather than one that changes strategy mid-game. This study uses Go differently. Go is a game of perfect information, with no unseen state or element of chance. Any difference in how two agents handle a mid-game change of opponent can be traced to their architecture and not to information they could not see.

2.2 Opponent-Induced Non-stationarity

2.2.1 Non-stationarity from a Changing Opponent. Standard reinforcement learning assumes a stationary environment with transition and reward dynamics fixed throughout learning [19]. An agent training against an opponent learns to exploit that opponent’s behaviour but if the opponent later plays differently those dynamics change and the agent’s gathered experience no longer describes the game in front of it [12]. This shifting environment is what is meant by non-stationarity and coping with it has been a recurring focus of reinforcement learning research [3, 13].

2.2.2 Why a Strategy Shift Breaks a Learned Policy. A policy is only as good as the conditions it was trained under. Through repeated game-play the agent settles on actions that worked well against the previous opponent, via value estimates that reflect how that opponent responded. When the opponent shifts, the same positions arise from different play, so actions once rewarded may no longer be and the agent’s value estimates stop matching what it now observes. Performance drops at the moment of the shift, recovering only once the agent has gathered enough experience of the new opponent to correct its policy [7].

2.3 Flat and Hierarchical RL

2.3.1 Flat Policies and PPO. The simplest way to build a learning agent is a flat policy, a single network that maps each observation directly to an action with no internal division into levels [2]. Proximal Policy Optimization (PPO) is a widely used method of this kind. It is an on-policy algorithm that improves the policy in small steps to keep learning stable [2]. This design is simple and effective but it holds all of the agent’s behaviour in one set of parameters. When the opponent shifts, the agent cannot change part of its behaviour while keeping the rest fixed. It has to revise the entire policy at once, risking overwriting useful skills. This is the weakness a hierarchical design tries to avoid.

2.3.2 Hierarchical Reinforcement Learning and Feudal Networks. Hierarchical reinforcement learning takes a different approach. It divides control across two levels of decision-making [15]. A high-level policy plans over the long term by setting sub-goals, while a low-level policy chooses the individual moves that work towards them. The low-level policy is guided by an intrinsic reward for reaching each sub-goal [11]. A sub-goal covers many moves, so the high level policy makes a lot less decisions than the low level policy makes moves. This compression of the problem is known as temporal abstraction [18]. Hierarchical methods split into two kinds. Options-based methods use a fixed set of temporally-extended actions [18] and subgoal-based methods such as h-DQN give the low-level policy a discrete state to reach [11]. Both need a set of options or subgoals defined in advance, which is hard in Go where no small, natural set exists. The Feudal Network (FuN) avoids this problem. Its manager sets a goal as a direction in a learned latent space rather than a discrete option or state, so the whole network trains end to end. A manager sets its goal at intervals and a worker acts at every step to follow it [21]. This separation also suits a variable opponent, since the manager can adopt a new plan once the opponent shifts while the worker keeps its low-level skills.

2.4 Measuring Adaptation and Recovery

The effect of a change in the environment is measured by how quickly an agent returns to its former level of performance, usually called recovery time or adaptation speed [1]. This relates to work on catastrophic forgetting, though the two differ. Forgetting concerns how much of a previously learned skill is lost when a network trains on a new task [10], while recovery concerns how quickly current performance returns once an opponent has changed. This study tracks performance as a rolling win rate over a fixed window of games, measuring recovery as the number of games the win rate stays below a set fraction of its pre-shift value. This is supported

by other measures of how far it falls and how much is lost while it stays there. These measures are defined in the experimental design to follow.

2.4.1 Recovery Under an Opponent Shift. This section has traced how a changing opponent creates non-stationarity. It has contrasted flat and hierarchical agents and shown how recovery is measured. These lines of work have not yet been brought together. How a hierarchical agent recovers when an opponent changes strategy has rarely been examined directly. Plus it has seldom been set against a flat agent under matched conditions. This study provides that comparison, using recovery from a controlled opponent shift in Go as its test. The next section sets how this comparison is designed and carried out.

3 Design and Implementation

This section describes how the research was built and run. Section 3.1 introduces the environment and the fixed opponents the agents were trained against. Section 3.2 presents the two agents under comparison, the flat PPO agent and the hierarchical Feudal Network. Section 3.3 sets out the experimental design, split into two phases. Phase 1 trains each agent to a stable baseline. Phase 2 introduces the abrupt opponent shifts and measures their effect. Section 3.4 defines the metrics used to assess recovery. Section 3.5 describes the tools and computing setup used to run the experiments.

3.1 Environment and Opponents

3.1.1 Environment. All experiments are run in `go_v5`, the Go environment from the PettingZoo library [20]. This provides a standard and reliable implementation of the complete rules of play. In each game the trained agent plays black and moves first, while the fixed opponent plays white. Two board sizes are studied as seen in Figure 1. The first is a 9×9 board with a komi of 5.5 and the second is a 13×13 board with a komi of 7.5. Komi adds a fixed number of points to the second player’s score to offset the advantage of moving first. These sizes balance strategic depth against the cost of training. This is because a smaller board shortens each game and allows many more to be played within a fixed budget of training steps.

At each turn the agent receives the board as a stack of seventeen binary planes of size $N \times N$, where N is the board dimension (9 or 13). These planes record the stones of both players over recent moves along with the colour to play [17]. The full observation encoding is illustrated in the appendix. For the agents own move it chooses among the $N \times N$ intersections or a single pass.

Two design choices ensure that every game ends in a clean and decisive result. First, the opponents are built to keep playing while valuable moves remain rather than passing early. In Go two passes end the game and the board is then scored exactly as it stands. If an opponent passed while territory was still contested it would end the game in an unfinished position. This would make the result depend on when it stopped rather than on the quality of play. By playing the position out, each opponent forces a full contest for territory, so the agent must genuinely control more of the board to win. Second, a fixed cap of $16N^2$ moves per game, which gives a possibility of 1296 moves on the 9×9 board and 2704 moves on the

13×13 board. The value was determined empirically by observing the longest natural game in practice. This prevents the rare case where two deterministic players could continue indefinitely under area scoring. When the cap is reached, both players are forced to pass. This makes the environment always return a final score. Together these choices give every game a clear area result, which provides the consistent performance signal the recovery analysis depends on.

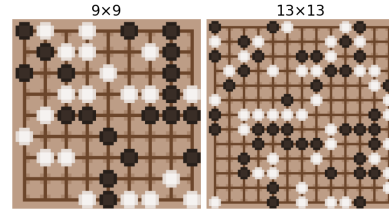


Figure 1: The two board sizes illustrated in PettingZoo `go_v5`.

3.1.2 Fixed Opponents. The agents are trained against a set of five fixed opponents, each following a simple hand-coded strategy. Every opponent plays deterministically and reacts only to the current board. This makes the behaviour stable and consistent. It also makes it possible to change opponents at certain points during training and to be confident any change in performance is from that switch.

The five styles differ in what they value. Further detail on each is given in the appendix. The greedy opponent takes the largest available area on each move and attacks only when a capture is offered. It builds thin, loosely held positions and does not hunt agent’s stones. The defensive opponent focuses on its own groups, connecting stones and protecting liberties while avoiding needless contact. The corner opponent plays logical contact moves with a lean towards one corner of the board. The edge opponent behaves similarly but leans towards the sides. The random opponent selects a legal move uniformly at random on every turn. It serves as a baseline with no strategy at all. The four strategic opponents are deliberately imperfect. They follow their chosen style but do not avoid basic tactical mistakes, which keeps each one beatable by a trained agent. Figure 2 shows example game positions alongside a move-density heatmap aggregated over many games. These together illustrate where each opponent characteristically plays.

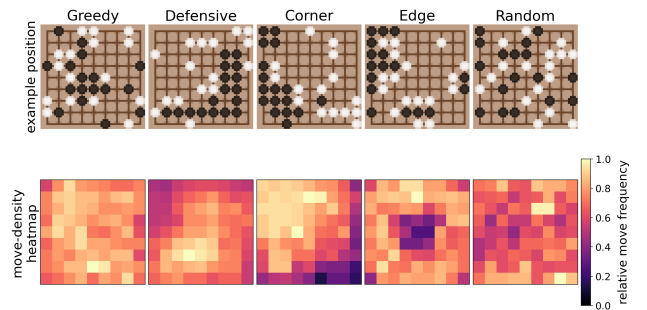


Figure 2: Each fixed opponent’s style, shown by example game positions and a move-density heatmap.

3.2 Agents

Two agents are compared, a flat agent and a hierarchical one. Both take the same board observation and pass it through the same first stage of processing. This stage is a convolutional neural network (CNN) that scans the board and builds a feature map of it [16]. The agents also share the same observation preprocessing. After this shared stage a further layer condenses the feature map into a single vector, which is its latent representation of the board. This is where the two agents start to differ. The hierarchical agent’s latent vector is considerably larger than the flat agent’s because its manager needs to hold more information to set its goals. Beyond this point the only difference between the two agents is the structure placed on top of this shared processing. Any difference in behaviour can then be traced to the architecture not to the input or the shared convolutional layers themselves. Figure 3 below shows a high-level view of the two agents’ architecture.

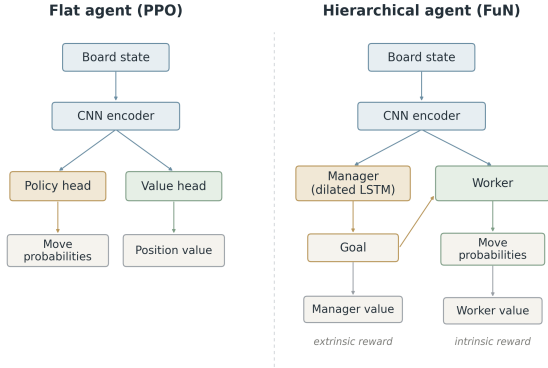


Figure 3: The two agents architecture.

3.2.1 Flat Agent (PPO). The flat agent is a single network trained with Proximal Policy Optimization [2].¹ From the CNN backbone’s internal representation, the network produces a probability distribution over possible moves and an estimate of how favourable the current position is, which guides learning. Illegal actions are removed before a move is sampled, so only legal placements or a pass can be selected [9]. Due to everything sitting in one network, a change in strategy has to be learned by updating the whole policy at once.

3.2.2 Hierarchical Agent (Feudal Network). The hierarchical agent is a Feudal Network [5, 21].² It shares the same backbone as the flat agent then splits into a manager and a worker. The manager acts less often and sets a goal as a direction in the network’s latent space. It uses a dilated long short-term memory network, known as a dilated LSTM. This retains information over many steps [8]. The worker acts every step, selecting moves that best follow the manager’s current goal. It is trained by an intrinsic reward measuring how closely the change in its state matches the direction the manager sets. This signal arrives every step, so the worker keeps learning even though the game’s reward comes only at the end.

¹The PPO implementation is adapted from <https://github.com/adi3e08/PPO>

²The Feudal Network implementation is adapted from <https://github.com/lweitekamp/feudalnets-pytorch>.

Several corrections were needed before the hierarchy trained correctly. In the implementation inherited at the start of the study, the manager’s goal was detached from the computation graph on every step. This removed the gradient the manager needed to learn useful goals, so the hierarchy stayed inactive. The same handling also limited the dilated memory to a single step. Once the goal gradient and the recurrent memory were restored, the manager began to set goals the worker could use. FuN’s win rate against every opponent recovered from near zero early in training and settled into a stable, competitive band, which is reported in the Phase-1 results. The key hyperparameters are listed in Table 1. PPO uses widely adopted default settings drawn from standard algorithm implementations [2]. The FuN values follow the Feudal Network [21] and the code reference implementation. A number of values were adjusted for stable training in the single-worker Go setting. Of the five training settings that both agents have in common, PPO kept the standard defaults for all. FuN matches PPO on the GAE λ and the entropy coefficient but uses a lower learning rate (2×10^{-4}), a shorter rollout length (400) and a higher manager discount (0.995) for stable training. The remaining rows of the table are specific to each agent’s own architecture. The two agents also differed in capacity, with FuN having far more parameters than PPO, roughly 26× on 9×9 and 47× on 13×13 (see appendix).

Table 1: Key hyperparameters.

Hyperparameter	PPO	FuN
<i>Shared</i>		
Learning rate	3×10^{-4}	2×10^{-4}
GAE λ	0.95	0.95
Entropy coefficient	0.01	0.01
Rollout length	2048	400
Discount γ	0.99	0.995 / 0.99
<i>PPO-specific</i>		
Minibatch size		256
Epochs per update		4
Clip range ϵ		0.2
<i>FuN-specific</i>		
Manager horizon c		15
Manager latent dim d		256
Worker embedding k		16
Intrinsic weight α		0.5

3.3 Experimental Design

Table 2 summarises the full experimental setup across both phases.

3.3.1 External Reference (GNU Go). The five heuristic opponents are tailor made, so they give no absolute sense of how strong the trained agents are. As a reference point, GNU Go, an established rule-based engine that plays Go without learning [6] is played against each opponent. It beats every opponent by a wide margin on both board sizes (see the appendix), fixing a strong upper reference. The trained agents are then read against this reference, showing whether each reaches the level of a competent engine on the same opponents or falls short.

3.3.2 Phase 1: Baseline Training. Phase 1 measures how well each agent can learn to play a single fixed opponent. Each agent is trained from scratch against each of the five opponents, one opponent per run. Both agents use the same step budget for every opponent and are evaluated with the same win rate criteria.

Table 2: Table of experiments.

	Phase 1	Phase 2
Purpose	Train agents to a stable per-opponent win-rate baseline	Measure recovery after abrupt opponent strategy shifts
Agents	PPO, FuN	PPO, FuN
Board sizes	9×9, 13×13	9×9, 13×13
Opponents	greedy, defensive, corner, edge, random	Shifting: corner→edge, corner→defensive, greedy→defensive.
Shift frequency	—	f1 (single), f2 (periodic), f3 (frequent)
Steps per run	3M (9×9), 5M (13×13)	2.3M (9×9), 4.0M (13×13)
Seeds	1	3
Runs	$2 \times 5 \times 2 = 20$	$3 \times 3 \times 2 \times 2 \times 3 = 108$
Recorded	rolling win rate (the baseline)	recovery time, dip depth, adaptation cost, disruption rate

Performance during training is tracked as a rolling win rate over recent games, which climbs as the agent learns the opponent and settles once further gains are small. The settled value is taken as the agent’s baseline against that opponent, setting the target for Phase 2 where the same agent must recover to it after the opponent changes.

3.3.3 Phase 2: Opponent Shifts. Phase 2 introduces the abrupt opponent shifts the study is built around. The complete pipeline is illustrated in the appendix. Each run resumes a stable Phase-1 checkpoint, so the agent begins from a known baseline against a known opponent. Midway through that opponent is replaced by a different one. Every shift is between two opponents that both agents can already beat, so a drop in performance reflects the disruption of the change itself rather than a jump to an unbeatable opponent. Training continues after the shift and the agent is left to recover on its own.

Shift magnitude. Three magnitudes are used, set by how far apart the two opponents’ styles are and how much disruption each shift causes. A LOW shift moves from the corner opponent to the edge opponent, both of which contest position but favour different regions of the board. A MED shift moves from corner to the defensive opponent, so the agent goes from facing an aggressive positioning style to a cautious one that consolidates its groups and avoids confrontation. A HIGH shift moves from the greedy opponent, which seizes open areas, to the defensive opponent that holds firm and refuses exchanges, making it the hardest shift to recover from. Further detail is given in the appendix.

Shift frequency. Frequency controls how often the opponent changes during a run. A single change tests plain recovery, while repeated changes test whether the agent can keep adapting when the environment will not settle. Three levels are studied, labelled f1, f2 and f3. f1 shifts once, isolating recovery from a single disruption. f2 shifts a few times at regular intervals, so the agent must recover more than once. f3 shifts many times in quick succession, leaving little time to settle between changes. Crossed with magnitude, this gives nine shift conditions, with the three shift schedules shown in the appendix.

Design. Each condition is run for both agents on both board sizes and repeated over three random seeds. This gives 108 Phase-2 runs. The seeds share the same Phase-1 starting point, so they capture the variability of Phase 2 rather than the initial training. Each run records the outcome of every game and recovery is measured from these records using the metrics defined next.

3.3.4 Parameters. Every run is given a fixed budget of environment steps, one step being a single move by the agent. Budgets differ by board size since the larger board is slower to learn. Phase 1 trains for 3 million steps and Phase 2 for 2.3 million on the 9 × 9 board, rising to 5 million and 4 million on 13 × 13, about 1.7× larger in each phase. Scaling both phases together gives an agent comparable time to reach its baseline and recover on either board, so recovery on the slower board is not cut off before it finishes.

A shift schedule lists the opponent faced and the exact step at which each change happens. Each Phase-2 run loads a Phase-1 checkpoint and follows its schedule while training continues. The first change is placed once the resumed agent’s rolling win rate has settled back to its Phase-1 level, which is at 300 000 steps on 9 × 9 and 500 000 on 13 × 13. The three frequencies then differ in how many changes follow. f1 makes a single change and switches back later, f2 makes a few evenly spaced changes and f3 makes ten changes in steady succession. Every schedule is fixed beforehand, so both agents and all seeds face the same sequence of changes at the same steps.

3.4 Metrics

Performance is measured throughout by a rolling win rate, which is the fraction of the last one hundred games won. A single game is too noisy on its own, so this rolling average gives a stable signal. The value just before a shift is taken as the pre-shift baseline and recovery is judged against a band set at 0.8 times that baseline. Three further measures describe how often a shift disrupts the agent and what happens once it does. Figure 4 marks all four measures on one real recovery segment.

Recovery time. Recovery time is the primary measure, applying the idea of adaptation speed from earlier work to a shifting opponent [1]. It is the number of games the rolling win rate stays below the 0.8 band, counted from the first drop below to the first return above it. The band is used as the rolling win rate fluctuates from game to game. A shorter recovery time means faster re-adaptation. If a shift never pushes the win rate below the band, recovery time is zero. If the win rate never climbs back above the band, the agent has not recovered and recovery time is continuously tracked to the end of that shift’s segment.

Dip depth. Dip depth measures how hard performance falls, the gap between the pre-shift baseline and the lowest rolling win rate reached after the shift. Large drops of this kind are a central concern when an agent must cope with a changing environment [13]. Dip depth captures severity rather than speed, so a shift can score a deep dip even when the agent recovers quickly.

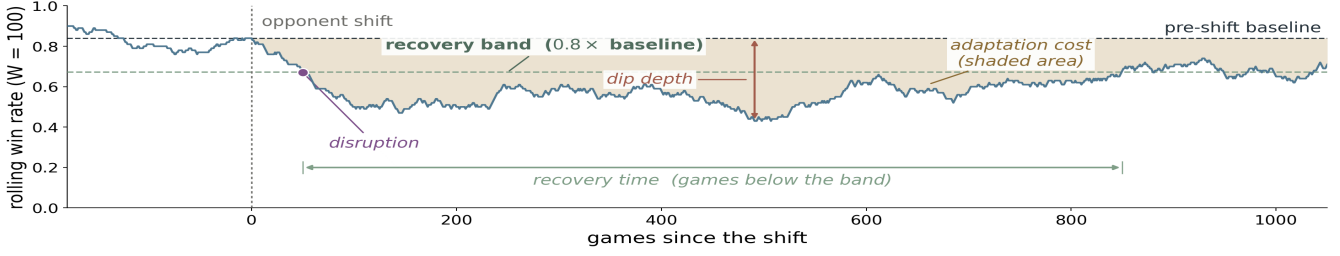


Figure 4: The four recovery measures on one real rolling win-rate segment.

Adaptation cost. Adaptation cost combines depth and duration into a single number. It is the total performance lost while the win rate sits below the baseline, which is summed in thousands of training steps across the post-shift segment. Measuring it in steps rather than games keeps it comparable across conditions whose games run to different lengths. This is a form of the cumulative loss an agent incurs while it re-adapts [19]. A deep, short dip and a shallow, long dip can carry the same cost. It gives an overall measure of the burden a shift imposes.

Disruption rate. A shift counts as a disruption if the rolling win rate drops below the 0.8 band at any point after it. The disruption rate is the number of shifts that count as a disruption, divided by the total number of shifts.

3.5 Tools

Both agents were implemented in PyTorch [14] and shared a training pipeline, which is run on two high-performance computing clusters. The University of Cape Town cluster provided GPU and CPU nodes, and the CHPC cluster provided additional CPU capacity. The full compute and software stack is summarised in the appendix. Each run produced a live metrics stream to Weights & Biases and a per-game log recording every game’s outcome. The per-game logs were the source for all recovery measures, computed offline after training. Each seed was run and stored separately then merged at analysis time for a mean and standard error, with three seeds used per Phase-2 condition. The full implementation is available online.³

4 Results

This section presents the results of the experiments set out in Section 3, covering both phases.

4.1 Phase-1 Capability

Table 3 reports the Phase-1 win rate of each agent against each opponent on both board sizes. PPO reaches a higher win rate than FuN in nine of the ten conditions, mostly above 90% and reaching 100% against two of the five opponents on the 9×9 board. FuN’s win rates are lower and more varied, ranging from 54% to 88%. The one exception is the edge opponent on the 13×13 board, where FuN’s win rate (87.0%) exceeds PPO’s (68.5%). GNU Go wins over 98% of games against every opponent on both boards (full results in the appendix) where PPO’s win rates sit close to this reference in most conditions. FuN’s remain further below it. The training

curves behind these final values are shown in Figures 6 and 7 for the 9×9 and 13×13 boards respectively. Raw values are shown faintly with an exponential moving average overlaid for clarity. PPO rises quickly and then keeps close to its ceiling, whereas FuN climbs more slowly and stays significantly noisier throughout training. The full set of per opponent training internals for both agents is provided in the appendix with curves also smoothed by an exponential moving average and raw values shown softly behind them. The complete interactive set of all runs is available on Weights & Biases.⁴

Figure 5 reports a second and independent measure of Phase-1 behaviour. It shows the average game length in environment steps for the same agent, opponent and board combinations. FuN’s games run longer than PPO’s in every one of the ten conditions, from about 4% longer (Defensive, 13×13) to over 3× longer (Corner, 13×13 and 9×9).

Table 3: Phase-1 final win rate (%).

Opponent	9 × 9		13 × 13	
	PPO	FuN	PPO	FuN
Greedy	99.0	75.5	92.5	74.0
Defensive	99.5	78.0	89.0	64.0
Corner	100.0	88.0	83.5	54.0
Edge	100.0	60.0	68.5	87.0
Random	98.0	63.5	98.0	83.0

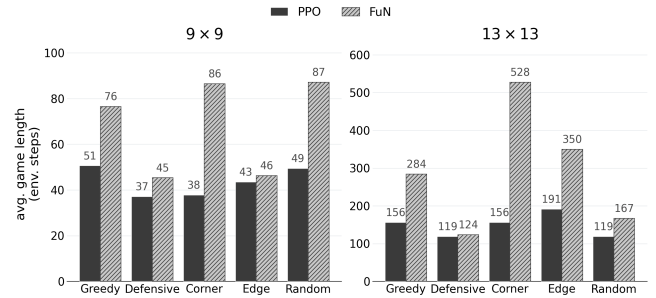


Figure 5: Phase-1 average game length (environment steps) by opponent and board size.

³https://github.com/bradscott/honours_rl_project

⁴<https://api.wandb.ai/links/bradscott4-university-of-cape-town/fct92j4k>

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

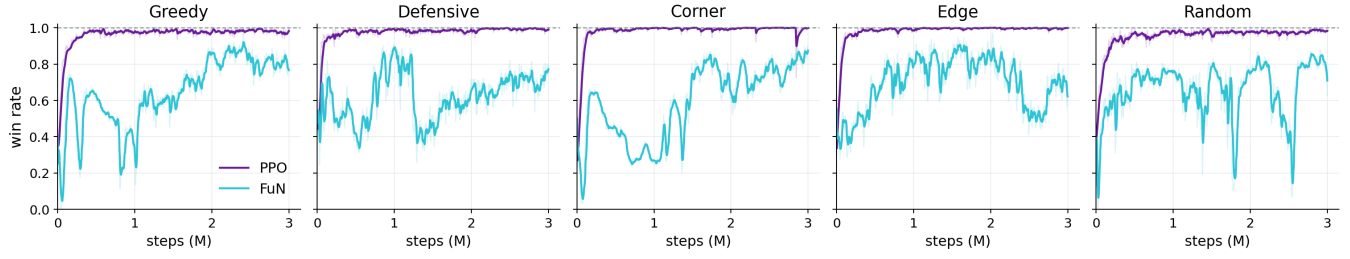


Figure 6: Phase-1 training curves on the 9×9 board.

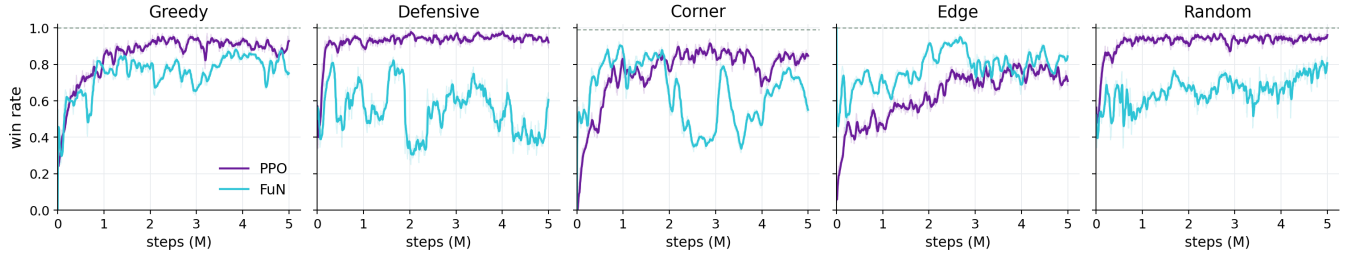


Figure 7: Phase-1 training curves on the 13×13 board.

4.2 Phase-2 Recovery

Table 4 reports Phase-2 recovery time, dip depth and adaptation cost for each agent. It is broken down by board size and shift magnitude as means over three seeds and the three shift frequencies (recovery time is also shown $\pm$ standard error).

Table 4: Phase-2 recovery time, dip depth and adaptation cost for PPO and FuN, by board size and shift magnitude.

Board	Mag.	Agent	Recovery (games)	Dip (win rate)	Adapt. cost (k steps)
9×9	Low	PPO	6 ± 4	0.07	2.8
		FuN	92 ± 37	0.23	18.9
	Med	PPO	22 ± 12	0.13	10.7
		FuN	316 ± 146	0.30	42.5
	High	PPO	18 ± 7	0.14	7.0
		FuN	693 ± 214	0.30	100.5
13×13	Low	PPO	0 ± 0	0.10	18.5
		FuN	21 ± 7	0.16	36.8
	Med	PPO	2 ± 1	0.11	17.3
		FuN	90 ± 51	0.24	76.3
	High	PPO	48 ± 25	0.17	19.4
		FuN	104 ± 59	0.27	120.2

Recovery time. PPO recovers faster than FuN in most board-magnitude combinations, though their standard error ranges overlap at 13×13 High. This is seen in Table 4 where it recovers 6-22 games versus FuN’s 92-693 games on 9×9 and 0-48 games versus 21-104 games on 13×13 . Figure 8 presents these recovery-time results as grouped bar charts, one panel per board.

Dip depth and adaptation cost. FuN’s win rate also drops further during a shift (dip depth 0.16–0.30, against PPO’s 0.07–0.17) and

costs more to recover (18.9–120.2k steps, against PPO’s 2.8–19.4k steps) across every condition shown in Table 4.

Rolling win-rate curves. Figures 9 and 10 show the rolling win-rate curves for both agents across all nine shift conditions on the 9×9 and 13×13 boards (shaded regions mark the post-shift opponent). PPO stays close to its pre-shift level throughout while FuN falls further and stays lower for longer. This is most visible at high magnitude with three shifts per run (bottom row, column f3).

Disruption rate. FuN is disrupted far more often than PPO in every board-magnitude combination. 52–75% of FuN’s shifts are disrupted, against PPO’s 0–27%. Figure 11 presents these disruption rates as a heatmap, one row per board and magnitude.

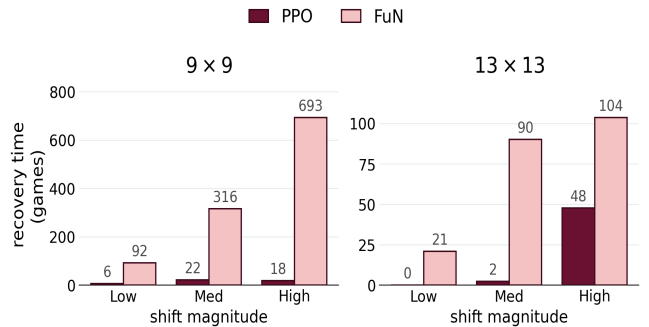


Figure 8: Mean recovery time (games) by shift magnitude.

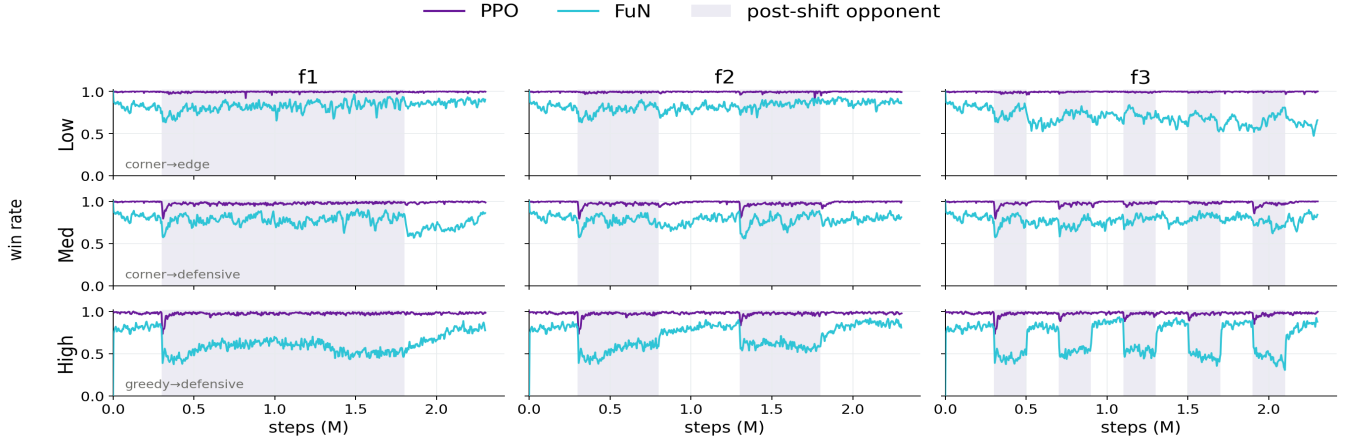


Figure 9: Rolling win-rate curves for PPO and FuN on the 9×9 board, across all nine shift conditions.

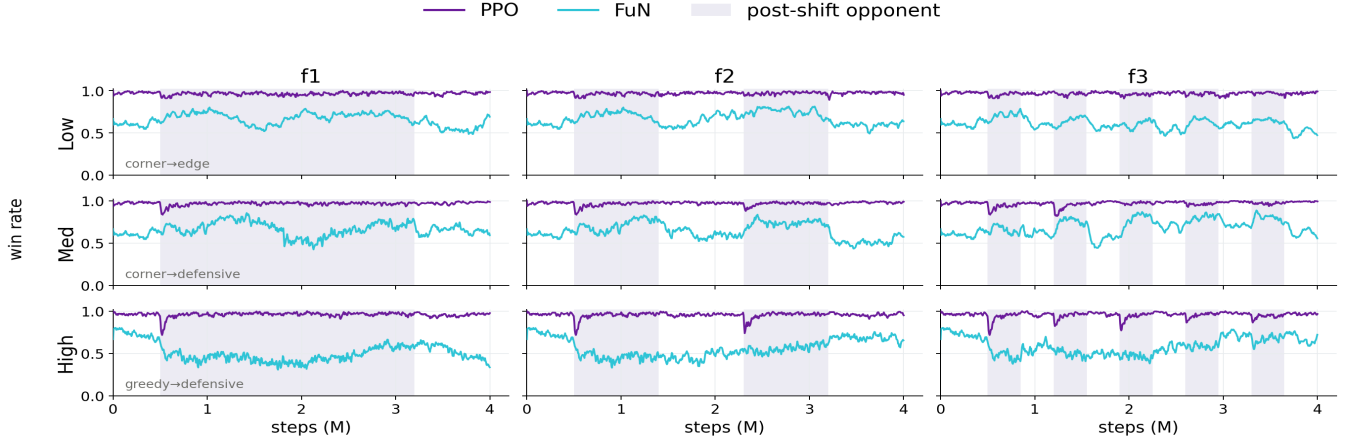


Figure 10: Rolling win-rate curves for PPO and FuN on the 13×13 board, across all nine shift conditions.

5 Discussion

This study compared a Feudal Network (FuN) agent against a flat PPO agent under controlled opponent strategy shifts in Go. Both agents were trained against five fixed opponents on 9×9 and 13×13 boards to a stable win-rate baseline. They then faced an abrupt mid-training opponent strategy change, which varied in magnitude and frequency. The research question asked whether hierarchical feudal reinforcement learning recovers faster than flat PPO after these shifts. The evidence gives a clear negative result. The flat agent recovered faster in most conditions tested. The subsections below examine this.

Phase-1 Baseline. Table 3 shows the hierarchical agent reached a weaker baseline in phase-1 training than the flat agent across all opponents on both board sizes, with win rates as low as 54% (Corner, 13×13) against PPO’s near-100%. The one exception is Edge on 13×13 (FuN 87.0% vs. PPO 68.5%), which is discussed in the Rolling Win-Rate Curves, Shift Magnitude and Frequency subsection. These weaker training win rates anticipate the hierarchical

agent’s greater fragility once shifts begin in Phase 2, indicating that the weaker baseline is an early sign of the same fragility seen in Phase-2 recovery. Evaluations of an architecture’s robustness to non-stationarity should treat a weak baseline as a warning sign of delicacy of control.

Phase-1 Game Length. FuN’s games ran longer than PPO’s in all ten Phase-1 conditions (Figure 5), from 4% longer against Defensive on 13×13 to over $3 \times$ longer against Corner on both boards. This reflects narrower winning margins and less capacity to absorb disruption. It anticipates the hierarchy’s slower recovery and higher disruption rate, which is reported in the Recovery Time and Disruption rate subsection. Recovery appears to depend not only on architecture but on the margin by which a policy already wins. This means decisiveness of victory, not win rate alone may be a precondition for an agent’s robustness in competitive settings. FuN’s longer games mean recovery time (measured in games) and adaptation cost (measured in steps) are not interchangeable units of experience.

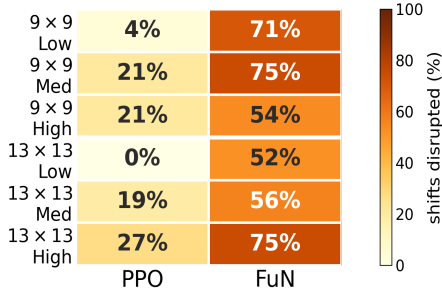


Figure 11: Fraction of shifts that disrupted each agent, by board and shift magnitude.

Both nonetheless point to the same conclusion: FuN recovers more slowly and at greater cost than PPO under every condition tested. This agreement across two measures with different units and different sensitivities to game length rules out game length itself as the explanation for the gap between the two agents. FuN’s longer games stretch each game relative to a step, so the two measures do not share a common scale. Recovery time counts games, while adaptation cost counts steps.

Phase-1 Training Curves. Figures 6 and 7 show that PPO’s win rate rises quickly against every opponent and holds close to its ceiling for the rest of training on both boards. FuN’s win rate climbs more slowly and continues to oscillate throughout, which never settles into a stable plateau. This instability is present before any opponent shift is introduced, so it precedes the conditions the research question addresses, yet still contributes to the gap this work sets out to close. Hierarchical agents trained on an intrinsic, self-generated reward signal may carry greater training variance than a flat baseline even in a fully stationary setting. A factor worth accounting for in any broader claim of adaptive advantage.

Recovery Time, Dip Depth and Adaptation Cost overview. Table 4 reports recovery time, dip depth and adaptation cost for both agents across every board and shift magnitude. Recovery time ranges from 0 to 48 games for PPO across both boards, against FuN’s 21 to 693 games. Dip depth follows the same pattern, reaching 0.07 to 0.17 for PPO against FuN’s 0.16 to 0.30. Adaptation cost ranges from 2.8 to 19.4 thousand steps for PPO, against FuN’s 18.9 to 120.2 thousand steps. PPO dips less deeply and costs less to recover than FuN in every board-magnitude combination as well as recovers faster than FuN in most of them. This does not support the expectation that the hierarchy recovers faster. The intended benefit of a hierarchy assumes that a shift can be absorbed by the manager alone, leaving the worker’s skills intact [21]. An opponent shift does not allow this. The worker faces different moves at every step, so both levels must re-adapt together [5]. On the 9×9 board, this leaves FuN needing between 92 and 693 games to recover against PPO’s 6 to 22 games. Manager-worker coupling therefore prevents the theorised separation of adaptation to the manager alone. A hierarchy of this kind may not contain the effect of a shift to a single level when the shift changes the environment’s dynamics rather than only its goal.

Rolling Win-Rate Curves, Shift Magnitude and Frequency. Figures 9 and 10 show the rolling win-rate curves for both agents across all nine shift conditions with the shaded region in each panel marking the post-shift opponent. PPO’s curve stays close to its pre-shift level throughout and shows only a brief dip after each shift before returning to its previous level. FuN’s curve behaves differently, falling more sharply at each shift with a visibly slower and less complete climb back. This decline grows with shift magnitude. At low magnitude, FuN’s trough is shallow and recovers reasonably quickly. At high magnitude, the trough is deeper with the climb back being slower and less complete. Shift frequency compounds this further. Under a single shift (f1), FuN’s curve shows one clear trough followed by recovery, but under frequent shifts (f3) each new shift arrives before the curve has recovered from the last, so the troughs continue and FuN’s win rate drifts downward across the run instead of returning to its earlier level. PPO shows no equivalent compounding under any frequency. One condition departs from this picture. Under the low-magnitude shift to the edge opponent on 13×13 , FuN’s curve dips only shallowly and recovers within the panel. This is closest to how PPO’s curves behave elsewhere. This is also the one Phase-1 condition in which FuN’s win rate exceeded PPO’s (Table 3). This prior competence appears to carry through into Phase 2. Where the worker’s underlying skill against the incoming opponent is already strong, the curve shows markedly less disruption. These rolling win-rate curves show that the hierarchy does not recover faster than the flat agent after an opponent shift. This disadvantage compounds rather than resolves under repeated shifts, with one exception where prior skill against the opponent is strong. An architecture’s response to a single shift may not predict its behaviour under a sequence of them. Repeated non-stationarity can expose resilience gaps a one-off recovery-time measurement would miss.

Recovery Time. Recovery time is the primary measure of the research question being tested and offers the clearest way to determine whether the hierarchy’s expected advantage extends to an opponent shift. Figure 8 presents these results as grouped bar charts, one panel per board. PPO’s bars remain low and flat across all three magnitudes on the 9×9 board, while FuN’s bar rises sharply to 693 games at high magnitude. On the 9×9 board, PPO recovers in 6 ± 4 games at low magnitude, 22 ± 12 at medium and 18 ± 7 at high, while FuN recovers in 92 ± 37 , 316 ± 146 and 693 ± 214 games respectively (Table 4). The gap widens sharply from low to high magnitude on this board size roughly fifteen times to more than thirty-eight times. On the 13×13 for contrast, PPO is never disrupted at low magnitude, it recovers in 2 ± 1 games at medium magnitude and 48 ± 25 games at high magnitude, while FuN recovers in 21 ± 7 , 90 ± 51 and 104 ± 59 games respectively. The gap is present at every magnitude on this board too, though it narrows considerably at high magnitude. This narrowing effect reflects the larger board’s longer games, which slow the movement of the rolling win-rate signal and compress recovery time as a measure. Dip depth and adaptation cost do not suffer this compression and remain informative on 13×13 where recovery time alone does not. PPO recovers faster than FuN in five of the six board-magnitude conditions with a clear margin but at 13×13 high magnitude the gap narrows enough that the two agents’ standard-error ranges overlap. This gives a direct and consistent

answer to the research question. Recovery time favours the flat architecture throughout. For competitive game-playing specifically, a hierarchy’s claimed advantage in adapting quickly to a new setting does not appear to generalise to an opponent that changes strategy mid-game. Published claims of fast hierarchical adaptation appear to be regime-specific. They hold for a fixed environment with a shifting goal and do not necessarily extend to adversarial, opponent-driven non-stationarity.

Dip Depth and Adaptation Cost. Dip depth and adaptation cost were introduced because recovery time compresses on the larger board, where longer games slow the movement of the rolling win-rate signal. Table 4 reports these values broken down by board and shift magnitude. FuN performs worse than PPO on both throughout. FuN’s win rate dips further during a shift, reaching 0.16 to 0.30 across all conditions, against PPO’s 0.07 to 0.17. FuN also costs more to recover between 18.9 and 120.2 thousand steps, against PPO’s 2.8 to 19.4 thousand steps. On 13×13 specifically, PPO’s adaptation cost stays close to flat across magnitude between 17 and 19 thousand steps, while FuN’s rises steadily from 37 to 76 to 120 thousand steps as magnitude increases. This pattern reflects what an opponent shift disturbs, unlike a task transfer. In a task transfer the worker’s skills stay valid, only the goal changes [21]. An opponent shift changes the moves the worker faces, so its low-level skill is no longer reliable. The hierarchy’s assumed protection does not hold. It does not dip less or cost less to recover than the flat agent in any condition tested and its disadvantage widens with magnitude rather than staying fixed. This reinforces the answer recovery time already gives to the research question. Where recovery time alone might be questioned on the larger board because its signal compresses as games lengthen, dip depth and adaptation cost confirm the same conclusion using measures that do not share that limitation. The widening adaptation cost with magnitude points to the same underlying cause behind the recovery-time result. Limiting an update to the manager alone works when a shift changes only the goal the agent pursues but not when it changes how the environment behaves.

Disruption Rate. Recovery time, dip depth and adaptation cost each describe how an agent behaves once it has been disrupted. None of them capture how often disruption happens in the first place. An agent with excellent average recovery numbers could still be disrupted on nearly every shift. Disruption rate closes this gap. It is the proportion of shifts that pushed an agent’s rolling win rate below its recovery band at any point, independent of how quickly or severely it recovered afterwards. Figure 11 presents the disruption rate as a heatmap, one row per board and shift magnitude, columns for each agent and colour indicating the percentage disrupted. The two agents occupy non-overlapping ranges. FuN’s disruption rate never falls below 52% (its minimum), while PPO never rises above 27% (its maximum), so even FuN’s best result is roughly double PPO’s worst. This gap is widest under the mildest shift tested, the condition where a robust architecture would be expected to absorb disruption most readily. At low magnitude, FuN is disrupted in 71% of shifts on 9×9 and 52% on 13×13 , against PPO’s at most 4%.

Contributions. This study provides the first direct, controlled comparison of a hierarchical Feudal Network and a flat PPO agent

recovering from an opponent-induced strategy shift in a competitive, perfect-information game [15, 21]. It provides a benchmark methodology built from shift schedules crossed with magnitude and frequency (Table 2), with four complementary metrics that avoid the blind spots any single measure carries. This design shows that the hierarchy’s adaptation advantage does not extend to opponent-driven non-stationarity, a pattern consistent across both boards and all three shift magnitudes. Instead, the flat agent dips less, costs less to recover and is disrupted less often in every condition tested as well as recovers faster in most of them (Table 4). For competitive game-playing, a hierarchy should not be adopted on the assumption that it is more robust to a changing opponent. A flat policy is the safer default unless other needs such as long-horizon planning justify a hierarchy. Temporal abstraction’s established benefits for task transfer and sample efficiency [11, 18] hold where only the goal changes. This study shows they do not automatically extend to an adversary that changes the dynamics themselves.

6 Conclusions

This report asked whether hierarchical feudal reinforcement learning recovers faster than flat PPO after abrupt opponent strategy shifts. It does not. The flat PPO agent recovered faster in most conditions, across both board sizes and all three shift magnitudes. On the 9×9 board it returned to its baseline within 6 to 22 games, against the Feudal Network’s 92 to 693 games. The larger 13×13 board narrowed this gap but never reversed it, with the flat agent recovering in as few as zero games while the hierarchy still needed up to 104 games. The complementary measures of disruption rate, dip depth and adaptation cost agreed with recovery time throughout. The main contribution of this study is the consistency of this result across every measure used, not any single number. The implication for reinforcement learning is that a manager-worker hierarchy’s effectiveness for task transfer and long-horizon control does not extend to an opponent that changes strategy. A shift of this kind disturbs the very dynamics the worker depends on, so the hierarchy cannot restrict the update to its manager. The structure intended to protect the agent instead left it vulnerable. Under the non-stationarity examined here, a hierarchy is not a substitute for a robust flat policy. Whether a different hierarchy design shielding its worker from these shifts or correcting its manager on a faster time scale could close this gap remains an open question. Until it is answered the simpler architecture remains the stronger choice.

7 Future work

This study’s Phase-2 seeds resume from a single Phase-1 run per condition, so widening the seed design to use separate Phase-1 runs per shift experiment would separate variance due to architecture from variance due to training. Recovery is judged by win rate, which records only whether a game was won, not by how much. A direct territory or score-based measure could replace it as the recovery signal, exposing disruption the present measure may absorb.

Acknowledgments

I thank my supervisor, Prof. Geoff Nitschke, for his guidance throughout this work. This research was supported by the University of Cape Town ICTS High Performance Computing Cluster team.

AI Declaration

Generative AI, specifically Claude, was used during the preparation of this report. The tools were used for brainstorming and planning the direction of the work, which helped shape the overall structure of the report and obtained feedback on specific sections.

References

- [1] Maruan Al-Shedivat et al. 2018. Continuous Adaptation via Meta-Learning in Nonstationary and Competitive Environments. In *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.
- [2] Marcin Andrychowicz et al. 2021. What Matters for On-Policy Reinforcement Learning? A Large-Scale Empirical Study. In *International Conference on Learning Representations (ICLR)*.
- [3] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. 2010. Multi-Agent Reinforcement Learning: An Overview. In *Innovations in Multi-Agent Systems and Applications - 1*, D. Srinivasan and L. C. Jain (Eds.). Studies in Computational Intelligence, Vol. 310. Springer, Berlin, Germany, pages: 183–221. doi:10.1007/978-3-642-14435-6_7
- [4] Murray Campbell et al. 2002. Deep Blue. *Artificial Intelligence* 134, 1-2 (2002), pages: 57–83. doi:10.1016/S0004-3702(01)00129-1
- [5] Peter Dayan and Geoffrey E. Hinton. 1993. Feudal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Free Software Foundation. 2009. GNU Go. Version 3.8.
- [7] He He et al. 2016. Opponent Modeling in Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 48)*, pages: 1804–1813.
- [8] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), pages: 1735–1780. doi:10.1162/neco.1997.9.8.1735
- [9] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the International Florida Artificial Intelligence Research Society Conference (FLAIRS)*, Vol. 35. doi:10.32473/flairs.v35i.130584
- [10] James Kirkpatrick et al. 2017. Overcoming Catastrophic Forgetting in Neural Networks. *Proceedings of the National Academy of Sciences* 114, 13 (2017), pages: 3521–3526. doi:10.1073/pnas.1611835114
- [11] Tejas D. Kulkarni et al. 2016. Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 29.
- [12] Michael L. Littman. 1994. Markov Games as a Framework for Multi-Agent Reinforcement Learning. In *Proceedings of the 11th International Conference on Machine Learning (ICML)*, pages: 157–163. doi:10.1016/B978-1-55860-335-6.50027-1
- [13] Sindhu Padakandla. 2022. A Survey of Reinforcement Learning Algorithms for Dynamically Varying Environments. *Comput. Surveys* 54, 6, Article 127 (2022), 25 pages. doi:10.1145/3459991
- [14] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32.
- [15] Shubham Pateria et al. 2021. Hierarchical Reinforcement Learning: A Comprehensive Survey. *Comput. Surveys* 54, 5, Article 109 (2021), 35 pages. doi:10.1145/3453160
- [16] David Silver et al. 2016. Mastering the Game of Go with Deep Neural Networks and Tree Search. *Nature* 529, 7587 (2016), pages: 484–489. doi:10.1038/nature16961
- [17] David Silver et al. 2017. Mastering the Game of Go without Human Knowledge. *Nature* 550, 7676 (2017), pages: 354–359. doi:10.1038/nature24270
- [18] Richard S. Sutton et al. 1999. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence* 112, 1-2 (1999), pages: 181–211. doi:10.1016/S0004-3702(99)00052-1
- [19] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [20] J. Terry et al. 2021. PettingZoo: Gym for Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34.
- [21] Alexander Sasha Vezhnevets et al. 2017. FeUdal Networks for Hierarchical Reinforcement Learning. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages: 3540–3549.

A Plots and Graphs

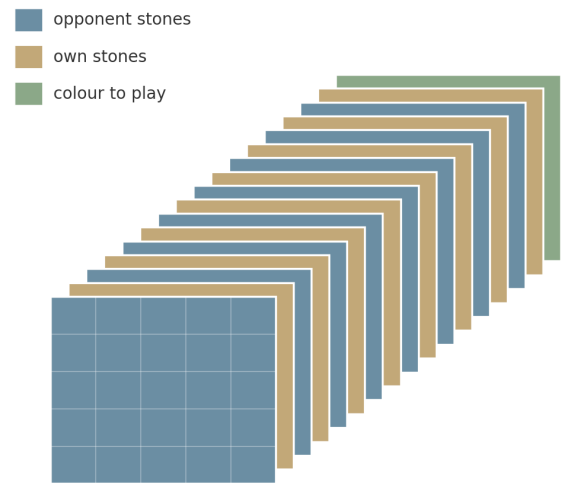


Figure 12: The go_v5 observation: eight times opponent and own stone planes plus a colour to play plane - seventeen in total.

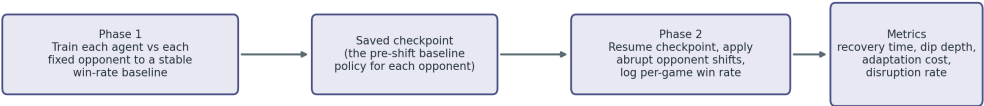


Figure 13: The two-phase pipeline.

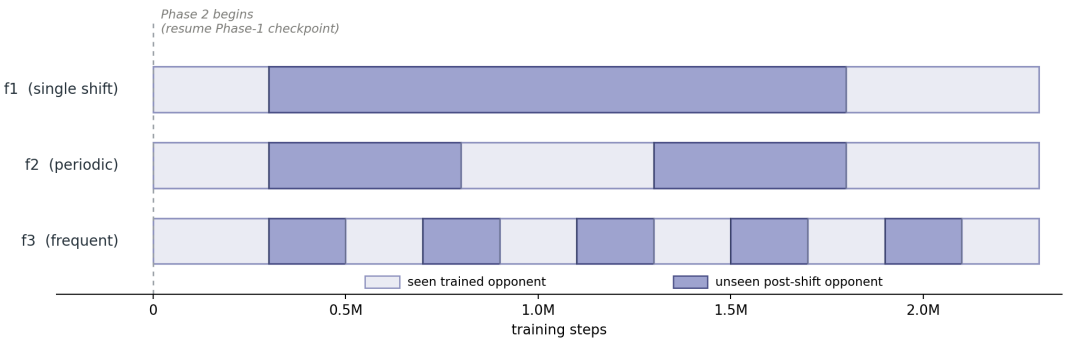


Figure 14: Phase-2 shift schedules for the three frequencies (9×9 , 2.3M steps).

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

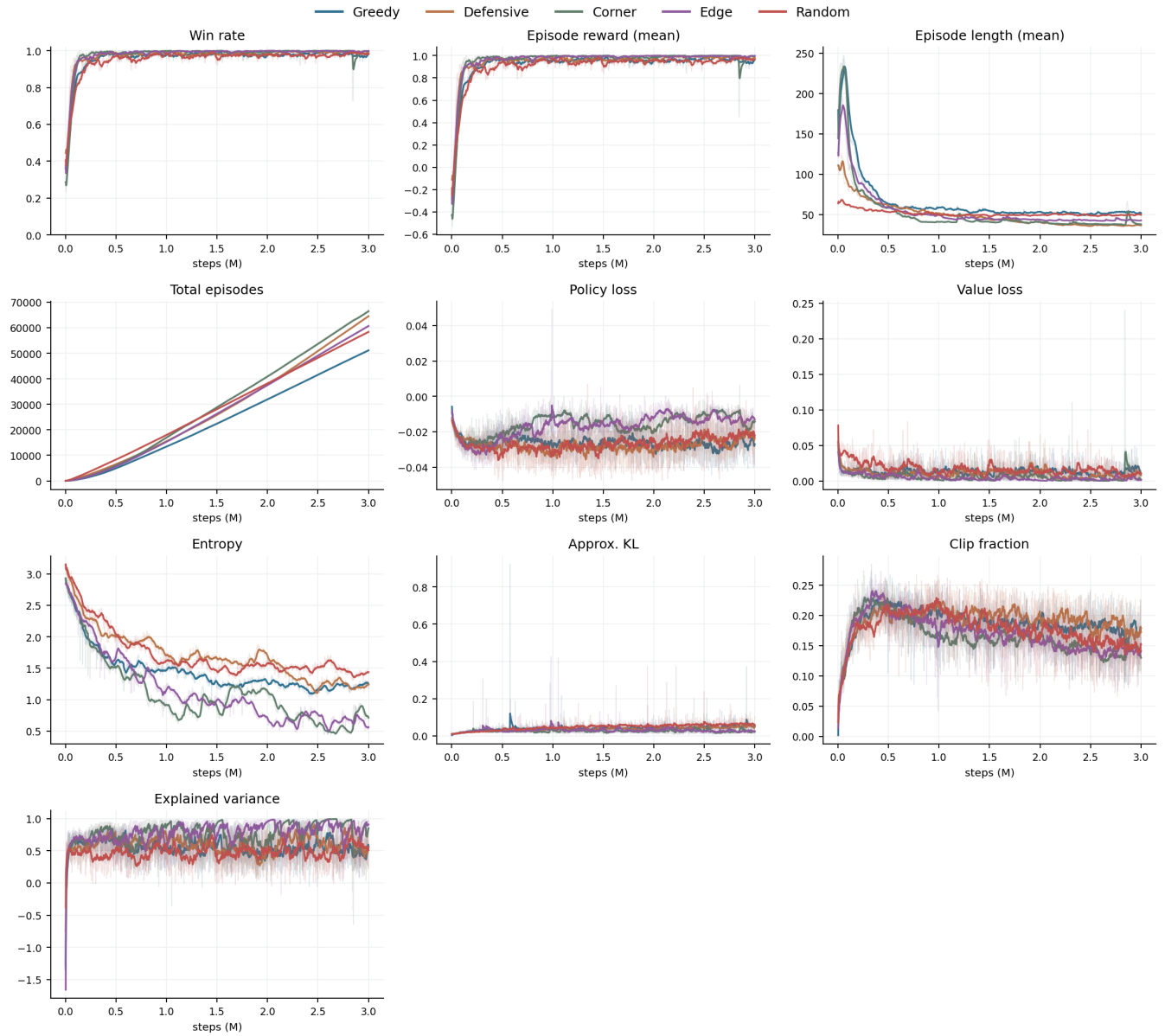
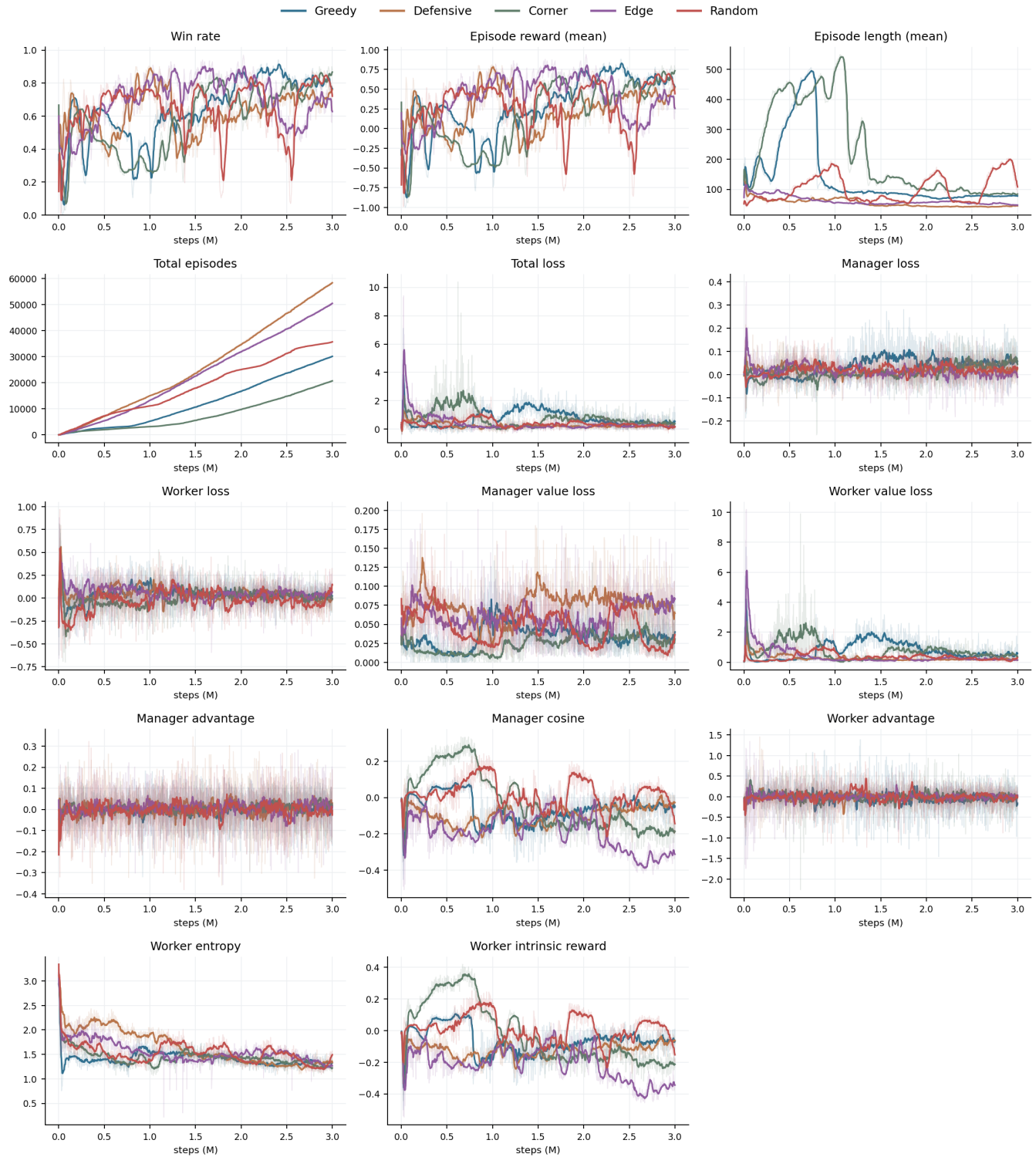


Figure 15: PPO Phase-1 training internals on the 9×9 board.

Figure 16: Feudal (FuN) Phase-1 training internals on the 9×9 board.

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

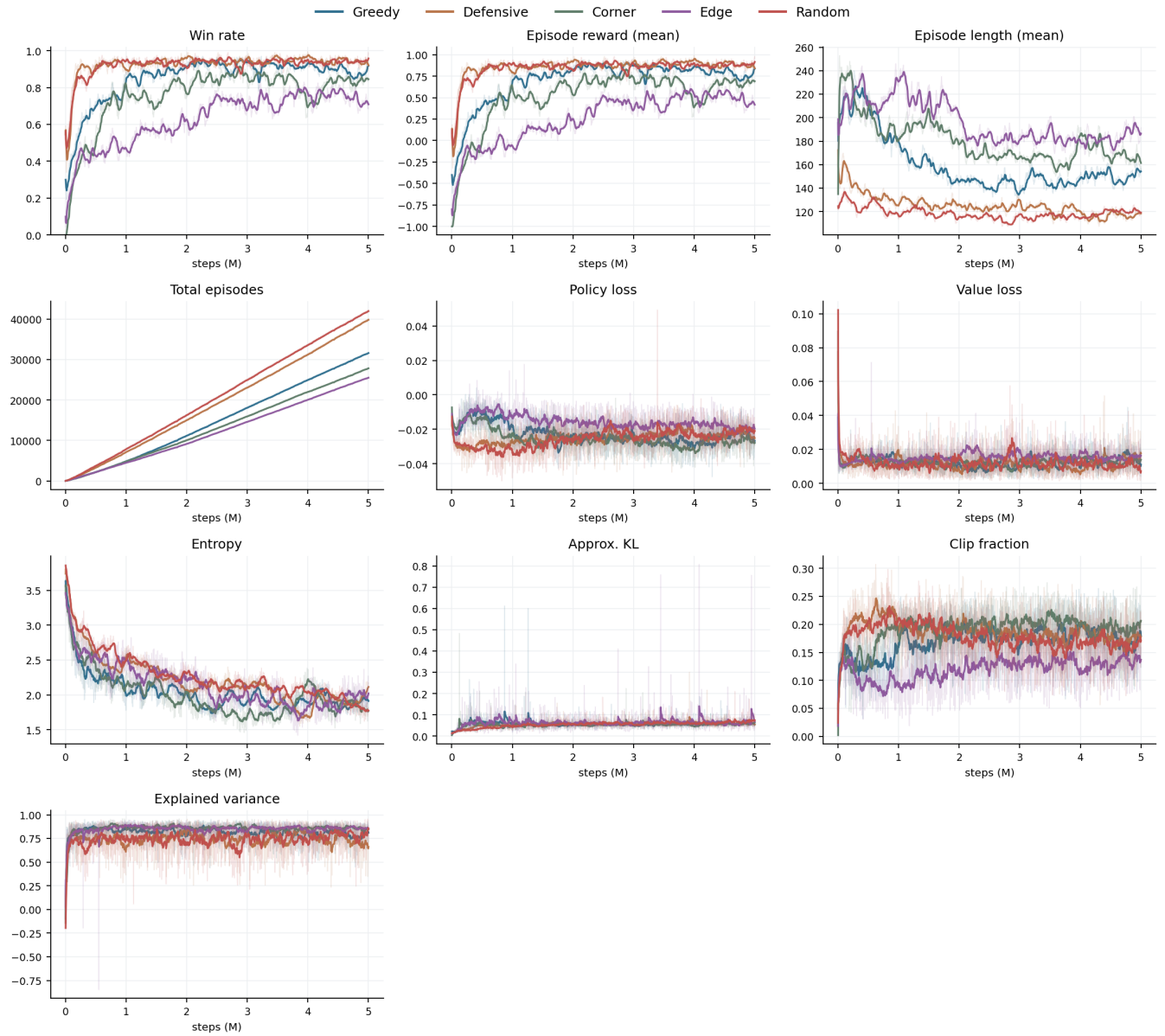


Figure 17: PPO Phase-1 training internals on the 13×13 board.

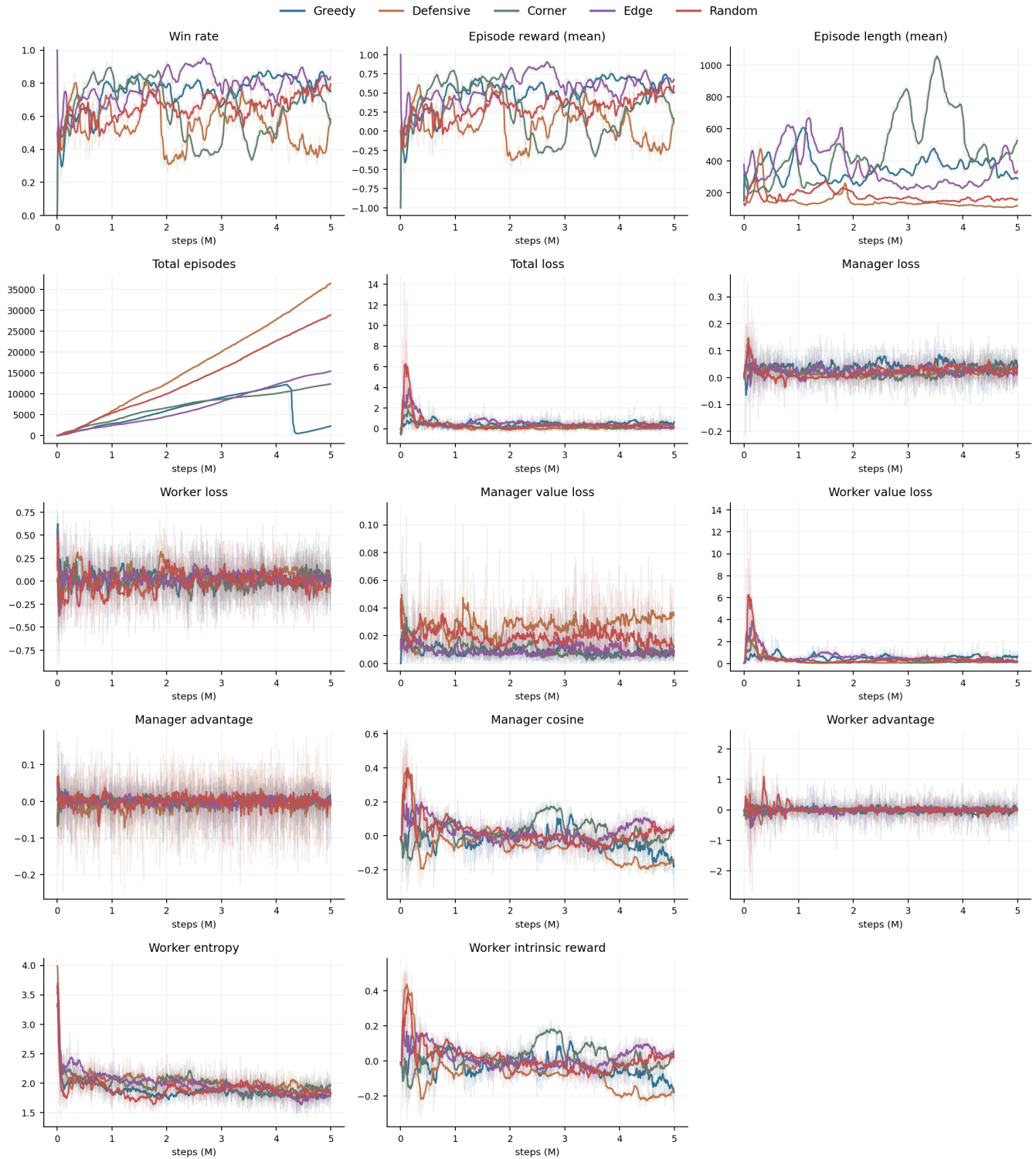


Figure 18: Feudal (FuN) Phase-1 training internals on the 13×13 board.

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

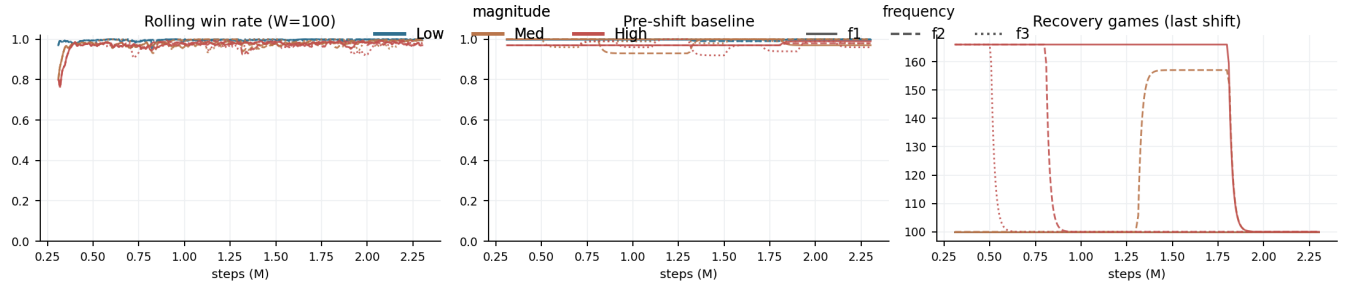


Figure 19: PPO Phase-2 recovery training on the 9×9 board.

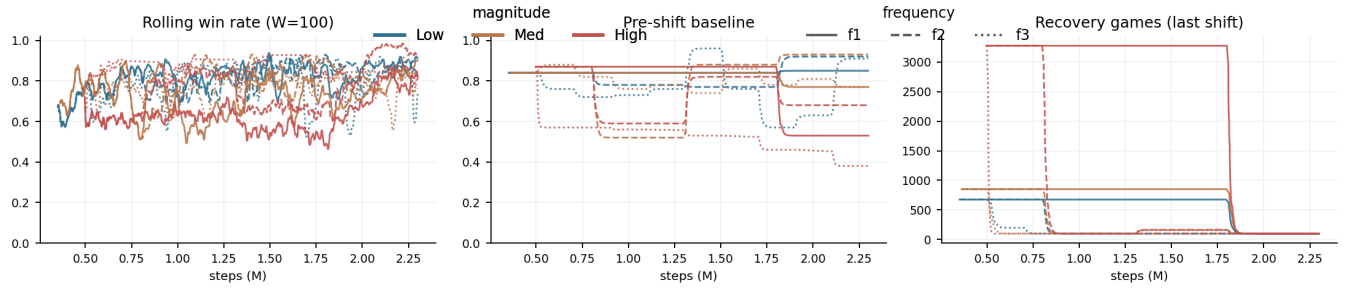


Figure 20: Feudal (FuN) Phase-2 recovery metrics on the 9×9 board.

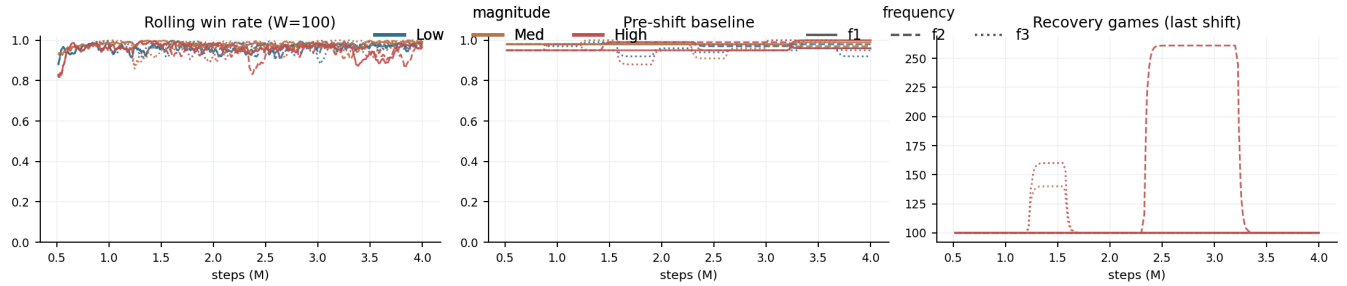


Figure 21: PPO Phase-2 recovery metrics on the 13×13 board.

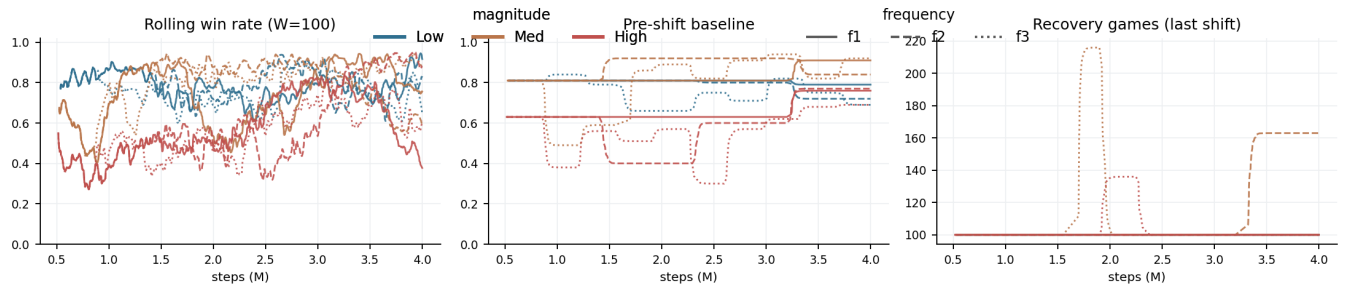


Figure 22: Feudal (FuN) Phase-2 recovery metrics on the 13×13 board.

B Tables

Magnitude	Pre-shift → Post-shift	Strategic change	Disruption
LOW	corner → edge	positional → positional	small
MED	corner → defensive	positional → group-defence	moderate
HIGH	greedy → defensive	territory-grabbing → group-defence	large

Table 5: Shift magnitude definitions.

Component	Parameters
<i>Shared</i>	
CNN conv trunk	0.023 M
<i>PPO (flat agent)</i>	
Latent projection (128-dim.)	0.332 M
Policy / actor head	0.011 M
Value / critic head	<0.001 M
Total (9×9)	0.37 M
Total (13×13)	0.74 M
<i>FuN (hierarchical agent)</i>	
Latent projection (256-dim.)	0.664 M
Manager (dilated LSTM + value)	0.59 M
Worker (LSTM + value + goal proj.)	8.31 M
Total (9×9)	9.59 M
Total (13×13)	34.5 M

Table 6: Parameter counts by component (M = Millions).

Opponent	GNU Go win rate (9×9)	GNU Go win rate (13×13)
Greedy	100%	100%
Defensive	100%	100%
Corner	100%	99%
Edge	100%	100%
Random	100%	100%

Table 7: GNU Go win rate against each opponent (level 10, 100 games each).

Opponent	Focus	Key behaviour
Greedy	Territory	Plays the largest open point each move.
Defensive	Group safety	Defends and connects its own groups and values liberties.
Corner	Contact + corner lean	Reasonable contact play with positional bias toward the corner.
Edge	Contact + edge lean	The same contact play with a positional bias toward the nearest edge.
Random	None (baseline)	Selects a uniformly random legal move each turn.

Table 8: The five fixed opponents key focus and behaviour.

Does hierarchical feudal reinforcement learning recover faster than flat PPO after abrupt opponent strategy shifts?

Component	Detail
<i>Compute</i>	
GPU	NVIDIA L40S (UCT HPC)
CPU	CPU nodes (UCT HPC and CHPC Lengau)
<i>Software</i>	
Python	3.10
PyTorch	2.11
PettingZoo (go_v5)	1.25
Gymnasium	1.2
NumPy	2.2
GNU Go	3.8
<i>Logging</i>	
Training metrics	Weights & Biases
Per-game records	CSV (games.csv)

Table 9: Compute and software stack.